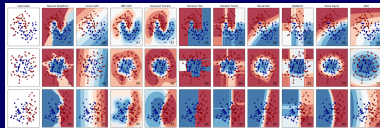


Reducción de dimensionalidad

Aprendizaje Automático · Sesión 5



Andrés Vásquez Restrepo

avasquezr3@eafit.edu.co

SI7009 - 1 (5553)

Universidad EAFIT

2026-2 – Medellín

Contenido

1. Por qué reducir dimensión
2. PCA
3. Más allá de lo lineal
4. t-SNE
5. UMAP
6. Evaluar un embedding
7. Hands-on y cierre

Dónde quedamos

Sesiones 1 a 4

- siempre hubo un target y : fraude, demanda, preferencia;
- el modelo se juzgaba contra ese y (PR-AUC, MASE, NDCG).

Hoy y mañana

- no hay y : la pregunta es **qué estructura tiene X** ;
- no hay una métrica obvia contra la cual comparar.

Idea clave: **Unidad 3**

Tabla $\rightarrow$ **representación** (S5) $\rightarrow$ grupos, anomalías y drift (S6).

Caso de las dos sesiones

Vendedores de **Olist**, contruidos con SQL sobre la misma base DuckDB del taller. El taller pide clientes: mismo método, otra unidad.

Por qué reducir dimensión

Una fila es un punto

	ventas	órdenes	ticket	flete/precio	categ.	retraso	días transp.	calif.
A	293	6	49	34 %	2	0 %	1,1	4,67
B	1.774	9	197	8 %	1	0 %	0,8	4,89
C	165.563	317	522	4 %	7	5 %	1,3	4,37

Cada vendedor es un vector $x \in \mathbb{R}^8$.

Parecido = cerca:

$$d(x, x') = \|x - x'\|_2 = \sqrt{\sum_j (x_j - x'_j)^2}$$

Sin preparar los datos

$d(A, B) \approx 1.481$ y $d(A, C) \approx 165.270$:
la distancia es casi solo **ventas**. Las
otras 7 columnas no cuentan.

1.740 vendedores con ≥ 3 órdenes entregadas antes de 2018-06-01.

La maldición de la dimensionalidad

Desde un punto, compare el vecino **más cercano** con el **más lejano**:

$$\frac{d_{\text{máx}} - d_{\text{mín}}}{d_{\text{mín}}}$$



- $p = 2$: el lejano está 130 veces más lejos;
- $p = 100$: solo un 32 % más lejos;
- $p = 1000$: un 11 %.

Idea clave

Con muchas dimensiones todos quedan casi a la misma distancia: "vecino" pierde sentido.

Cuatro usos de la reducción

Uso	Método típico	Cómo se evalúa
Visualizar	t-SNE, UMAP, PCA 2D	¿Se conservan los vecinos? ¿Es estable?
Comprimir	PCA	Varianza explicada, error de reconstrucción
Quitar ruido y colinealidad	PCA	Desempeño del modelo siguiente
Preprocesar para clustering o modelos	PCA, UMAP a 5–10 dims	Métrica de la tarea siguiente, con CV

Idea clave

No hay “mejor reducción” en abstracto: hay mejor reducción **para un uso**.

Selección frente a extracción

Definición: Seleccionar variables

Quedarse con un subconjunto de las columnas originales.

- + interpretables: "ventas" sigue siendo ventas;
- pierde la información que solo existe en combinaciones;
- dos columnas redundantes siguen ahí.

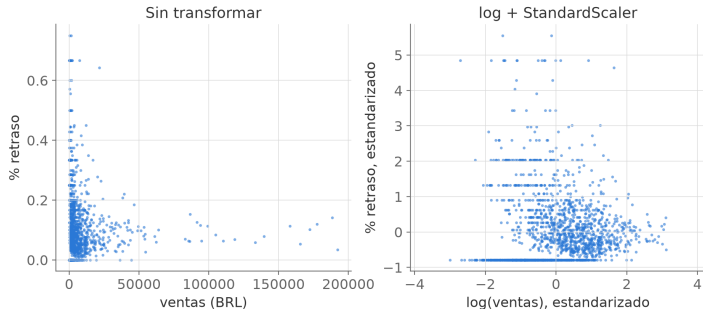
Definición: Extraer variables nuevas

Construir $z = f(x)$ con menos dimensiones.

- + compacta la información de muchas columnas;
- + elimina redundancia;
- cada z_j hay que interpretarla (PCA) o no se puede (t-SNE).

Hoy: **extracción**. Lineal (PCA) y no lineal (t-SNE, UMAP).

El escalado decide la geometría

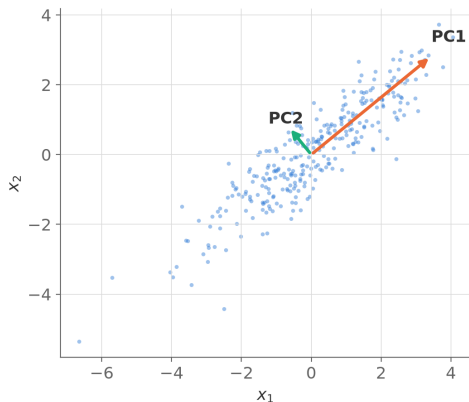


- ventas va de 30 a 500.000 BRL: domina cualquier distancia;
- $\log_{10}$ para variables sesgadas (ventas, órdenes, ticket, categorías, días);
- luego StandardScaler: media 0, desviación 1.

En no supervisado el pre-procesamiento **es** parte del modelo: otra transformación, otros vecinos.

PCA: proyección lineal

La idea



1. centrar los datos;
2. buscar la dirección donde la nube **se estira más** (PC1);
3. la siguiente dirección, perpendicular a la anterior, que más se estira (PC2), y así;
4. proyectar cada punto sobre las primeras k direcciones.

Idea clave

PCA es una **rotación** de ejes seguida de quedarse con los primeros k . Nada más.

El problema de optimización

Con X centrada y $\Sigma = \frac{1}{n-1} X^\top X$, la varianza de la proyección $z = X\mathbf{w}$ es $\mathbf{w}^\top \Sigma \mathbf{w}$:

$$\mathbf{w}_1 = \arg \max_{\|\mathbf{w}\|=1} \mathbf{w}^\top \Sigma \mathbf{w}$$

Con un multiplicador de Lagrange, $\mathcal{L} = \mathbf{w}^\top \Sigma \mathbf{w} - \lambda(\mathbf{w}^\top \mathbf{w} - 1)$, y derivando:

$$\Sigma \mathbf{w} = \lambda \mathbf{w} \quad \Rightarrow \quad \text{Var}(X\mathbf{w}) = \mathbf{w}^\top \Sigma \mathbf{w} = \lambda$$

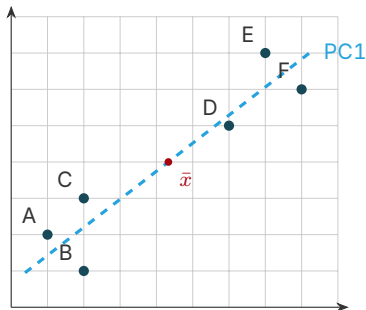
Definición: Componentes

Los **eigenvectores** de Σ , ordenados por eigenvalor.

Definición: Varianza de cada componente

El **eigenvalor** λ_j . Suman la varianza total.

PCA a mano: el mini-mundo



1. media: $\bar{x} = (4,33; 4,00)$
2. covarianza: $\Sigma = \begin{pmatrix} 9,07 & 6,60 \\ 6,60 & 5,60 \end{pmatrix}$
3. eigenvalores: $\lambda_1 = 14,16, \lambda_2 = 0,51$
4. PC1: $w_1 \approx (0,79; 0,61)$
5. PC1 explica $\frac{14,16}{14,67} = \mathbf{96,5\%}$

	A	B	C	D	E	F
z_1	-3,86	-3,68	-2,46	1,93	3,94	4,12

Un solo número por punto conserva casi todo: separa A, B, C de D, E, F. (Mañana volvemos a estos 6 puntos.)

PCA vía SVD

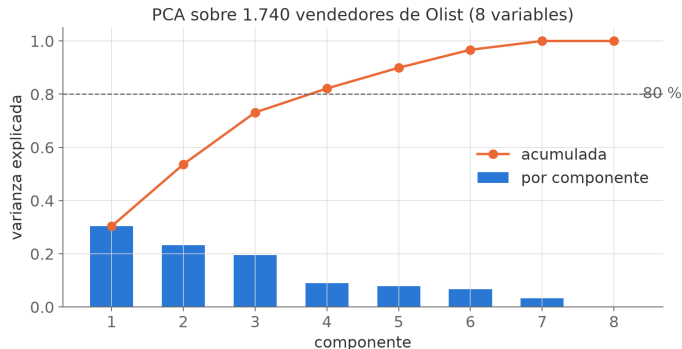
$$\begin{array}{|c|} \hline X_c \\ \hline n \times p \\ \hline \end{array} = \begin{array}{|c|} \hline U \\ \hline n \times p \\ \hline \end{array} \begin{array}{|c|} \hline S \\ \hline \text{diag.} \\ \hline \end{array} \begin{array}{|c|} \hline V^\top \\ \hline p \times p \\ \hline \end{array}$$

- componentes principales = filas de $V^\top$ (columnas de V);
- varianzas: $\lambda_j = s_j^2 / (n - 1)$;
- coordenadas: $Z = X_c V_k = U_k S_k$.

Por qué las librerías usan SVD y no Σ

No forma $X^\top X$ (pierde precisión numérica). Con n o p grandes, `svd_solver='randomized'` calcula solo los primeros k componentes.

Varianza explicada y scree plot



$$\text{EVR}_j = \frac{\lambda_j}{\sum_l \lambda_l}$$

- PC1: 30 %, PC2: 23 %, PC3: 20 %;
- 4 componentes → 82 %;
- PC8 ≈ 0 : $\log(\text{ventas}) \approx \log(\text{órdenes}) + \log(\text{ticket})$. Redundancia exacta.

Idea clave

Criterios: 80–90 % acumulado, el codo, o el k que mejor sirve a la tarea siguiente.

Leer los loadings

ventas	0.61	-0.09	0.08
nº órdenes	0.43	-0.34	0.37
ticket medio	0.41	0.35	-0.41
flete / precio	-0.42	-0.24	0.44
nº categorías	0.31	-0.29	0.33
% retraso	0.03	0.49	0.35
días a transportadora	0.02	0.42	0.32
calificación	-0.06	-0.44	-0.41
	PC1 (30%)	PC2 (23%)	PC3 (19%)

PC1 (30 %): tamaño del vendedor

+ ventas, órdenes, ticket, categorías; – flete relativo.

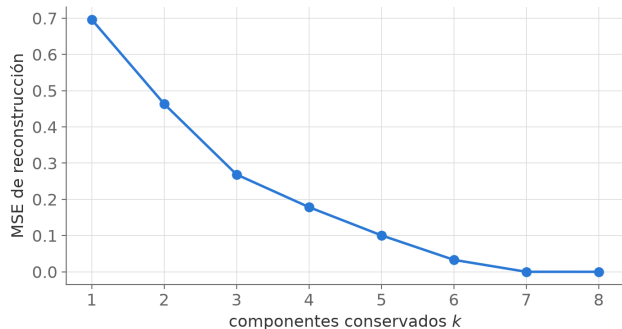
PC2 (23 %): mala experiencia de entrega

+ % retraso, días a la transportadora; – calificación.

Cuidado

El signo es arbitrario. Nombrar un componente es una **hipótesis** que hay que defender con los datos.

Reconstrucción



Volver al espacio original con k componentes:

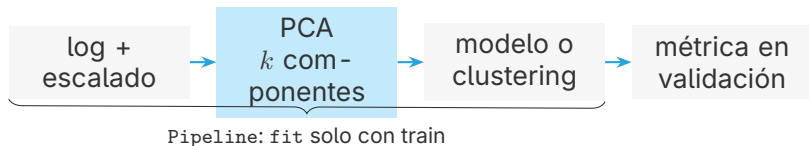
$$\hat{x} = \bar{x} + V_k V_k^\top (x - \bar{x})$$

- el error baja con k y es 0 con $k = p_i$
- con $k = 4$: MSE = 0,18 (sobre datos estandarizados).

Idea clave: Puente a la S6

Quien **se reconstruye mal** no se parece al resto: el error sirve como score de anomalía.

PCA dentro del pipeline



Leakage, otra vez (S1)

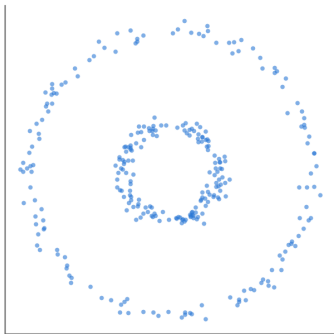
Ajustar PCA o el escalador con todos los datos usa información del test para construir las features.

Idea clave: Hiperparámetro

k se elige con CV como cualquier otro hiperparámetro (S2): `pca__n_components` dentro de la búsqueda.

Cuándo PCA falla

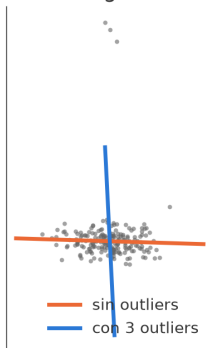
No lineal: anillos



No lineal

ninguna recta separa los
anillos

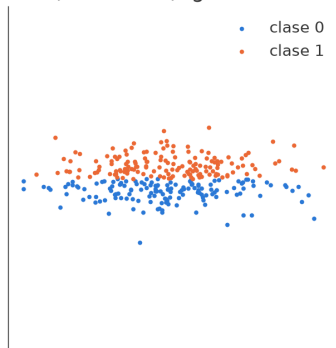
Outliers giran PC1



Outliers

3 puntos giran PC1 casi 90°

PC1 (horizontal) ignora la clase



Varianza $\neq$ relevancia

lo que separa las clases está
en PC2

PCA en código

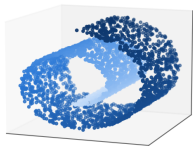
```
1 from sklearn.pipeline import make_pipeline
2 from sklearn.preprocessing import (
3     FunctionTransformer, StandardScaler)
4 from sklearn.decomposition import PCA
5
6 pca_pipe = make_pipeline(
7     FunctionTransformer(np.log1p),
8     StandardScaler(),
9     PCA(n_components=0.9), # 90 % var.
10 )
11 Z = pca_pipe.fit_transform(X_sellers)
12
13 pca = pca_pipe[-1]
14 pca.explained_variance_ratio_
15 pca.components_ # loadings
16 pca.inverse_transform(Z) # reconstruccion
```

- `n_components` entero = k ;
fracción = varianza mínima;
- `components_` tiene forma (k, p) : una fila por componente;
- aplique `log1p` solo a las columnas sesgadas (`ColumnTransformer`).

Más allá de lo lineal

La hipótesis de la variedad

Swiss roll en 3D



PCA 2D: sigue enrollado



UMAP 2D: desenrolla (por tramos)



- los datos de alta dimensión suelen vivir cerca de una **superficie** de pocas dimensiones;
- en el swiss roll, dos puntos cercanos en línea recta pueden estar lejos **sobre la superficie**;
- PCA solo rota y proyecta: deja la espiral enrollada.

Global frente a local

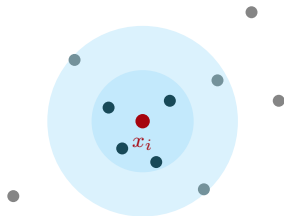
Método	Qué preserva	Qué distorsiona
PCA	varianza global, distancias grandes	estructura no lineal, vecindarios finos
Kernel PCA, Isomap	estructura no lineal global	costo $O(n^2)$ o más; sensibles a parámetros
t-SNE	vecindarios locales	distancias entre grupos, tamaños, densidades
UMAP	vecindarios locales y algo de lo global	distancias globales (menos que t-SNE)

Idea clave

En 2D no cabe todo. Cada método **elige qué sacrificar**. Antes de leer una figura hay que saber qué sacrificó.

t-SNE

Vecindades como probabilidades



vecinos cercanos: $p_{j|i}$ alta

En el espacio original, la probabilidad de que x_i "elija" a x_j como vecino:

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

Simetrizada:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

Idea clave

t-SNE convierte distancias en una distribución de "quién es vecino de quién".

Perplexity

Cada punto tiene su propio ancho σ_i . Se ajusta para que la entropía de P_i dé una perplexity fija:

$$\text{Perp}(P_i) = 2^{H(P_i)}, \quad H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}$$

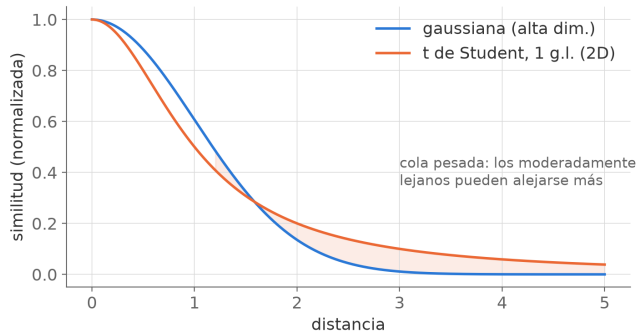
Definición: **Lectura**

Perplexity $\approx$ **número efectivo de vecinos** que mira cada punto. Valores típicos: 5 a 50.

Definición: **Densidad adaptativa**

En zonas densas σ_i es pequeño; en zonas ralas, grande. Por eso t-SNE **borra** las diferencias de densidad.

Student-t y el problema de aglomeración



En 2D no caben todos los vecinos de un punto de alta dimensión: si se usa otra gaussiana, todo se amontona en el centro.

En el embedding y :

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

Idea clave

La cola pesada deja que puntos moderadamente lejanos se separen mucho. Por eso aparecen "islas".

El objetivo: divergencia KL

$$C = \text{KL}(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Asimétrica

p_{ij} alto y q_{ij} bajo (separar vecinos): **costo alto**.

p_{ij} bajo y q_{ij} alto (juntar lejanos): **casi gratis**.

Optimización

Descenso de gradiente sobre las y_i ; *early exaggeration* al inicio; inicializar con PCA da resultados más estables.

Idea clave

t-SNE se esfuerza por mantener juntos a los vecinos. No le importa dónde quedan los grupos entre sí.

Mismos datos, distinta perplexity

perplexity = 5



perplexity = 30



perplexity = 50



perplexity = 100



1.740 vendedores, mismas 8 variables, color = $\log_{10}(\text{ventas})$. Cambia el tamaño de las "islas", su forma y la distancia entre ellas.

Si una conclusión cambia con la perplexity, no era una conclusión sobre los datos.

Lecturas permitidas y prohibidas

Se puede leer

- quién es vecino de quién;
- gradientes de una variable a lo largo del mapa;
- puntos aislados para inspeccionar.

No se puede leer

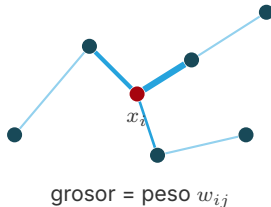
- distancia entre grupos;
- tamaño o densidad de un grupo;
- número de grupos;
- ejes (no significan nada).

Limitaciones prácticas

No tiene transform: un punto nuevo obliga a recalcular todo. Costo $O(n \log n)$ con Barnes-Hut. Pensado para visualizar en 2D o 3D, no para crear features.

UMAP

Un grafo de vecinos difuso



1. buscar los k vecinos más cercanos de cada punto;
2. pesar cada arista:

$$w_{ij} = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right)$$

ρ_i = distancia al vecino más cercano (el más cercano siempre pesa 1);

3. simetrizar: $w = w_{ij} + w_{ji} - w_{ij}w_{ji}$.

Idea clave

Igual que t-SNE, cada punto mide distancias con su propia escala local.

Optimizar el layout

Buscar posiciones y_i en 2D cuyo grafo se parezca al original, minimizando una cross-entropy:

$$C = \sum_{(i,j)} \underbrace{w_{ij} \log \frac{w_{ij}}{v_{ij}}}_{\text{atracción}} + \underbrace{(1 - w_{ij}) \log \frac{1 - w_{ij}}{1 - v_{ij}}}_{\text{repulsión}}, \quad v_{ij} = \left(1 + a\|y_i - y_j\|^{2b}\right)^{-1}$$

Atracción

Sobre las aristas del grafo: vecinos se acercan.

Repulsión

Sobre pares al azar (*negative sampling*): no vecinos se alejan. Esto la hace rápida.

A diferencia de KL, el segundo término también castiga juntar lo que estaba lejos: UMAP conserva algo más de estructura global.

`n_neighbors` y `min_dist`

`n_neighbors=5, min_dist=0.0`



`n_neighbors=5, min_dist=0.3`



`n_neighbors=5, min_dist=0.9`



`n_neighbors=50, min_dist=0.0`



`n_neighbors=50, min_dist=0.3`



`n_neighbors=50, min_dist=0.9`



Definición: `n_neighbors`

Pocos: estructura muy local, el mapa se rompe en hilos. Muchos: más global, más suave.

Definición: `min_dist`

Qué tan apretados pueden quedar los puntos. 0 = grumos densos (útil para clustering).

Color = $\log_{10}(\text{ventas})$.

UMAP frente a t-SNE

	t-SNE	UMAP
Estructura global	poca	algo más
Datos nuevos	recalcular todo	<code>transform()</code>
Dimensiones de salida	2 o 3	cualquiera (p. ej. 10)
Métricas de distancia	euclídea (en la práctica)	cualquiera (coseno, Jaccard...)
Velocidad con n grande	lenta	rápida
Determinista	no	no

Idea clave

Para visualizar, los dos sirven. Para usar el embedding como entrada de otra cosa, UMAP.

UMAP antes de clustering

Práctica común

- UMAP a 5–10 dimensiones (no 2);
- `min_dist=0` para grumos densos;
- luego HDBSCAN (mañana).

Funciona bien con texto, imágenes y embeddings de muchas dimensiones.

Riesgo

UMAP puede **crear** separaciones que no existen en los datos: grupos “limpios” que en el espacio original se solapan.

Idea clave: Regla

Agrupar en el embedding está permitido; **validar y perfilar** los grupos se hace en las variables originales.

Evaluar un embedding

Lo visual no basta

Antes de mirar la figura, responda:

1. **¿Qué variables?** Un embedding de ventas y órdenes muestra tamaño, no comportamiento.
2. **¿Qué escalado?** Sin log ni estandarización, una columna decide todo (R5).
3. **¿Qué parámetros?** perplexity, n_neighbors, min_dist.
4. **¿Qué semilla?** ¿Y qué pasa con otras?

Una figura sin estos cuatro datos no es reproducible, y lo que no se puede reproducir no se puede defender.

Las tres slides siguientes convierten "se ve bien" en números.

Trustworthiness y continuity

Con k vecinos y $r(i, j)$ = rango de j entre los vecinos de i en el espacio original:

$$T(k) = 1 - \frac{2}{nk(2n - 3k - 1)} \sum_{i=1}^n \sum_{j \in U_k(i)} (r(i, j) - k)$$

$U_k(i)$: vecinos de i en el embedding que **no** lo eran en el original.

Definición: Trustworthiness

Penaliza **falsos vecinos**: ¿puedo confiar en lo que veo junto? 1 = perfecto.

`sklearn.manifold.trustworthiness(X, Y, n_neighbors=10)` (Venna y Kaski, 2001)

Definición: Continuity

La simétrica: penaliza **vecinos perdidos**. ¿Lo que estaba junto sigue junto?

Preservación de vecinos en Olist

Fracción de los 10 vecinos originales de cada vendedor que siguen entre sus 10 vecinos en 2D:

$$\text{kNN}(k) = \frac{1}{n} \sum_i \frac{|N_k^X(i) \cap N_k^Y(i)|}{k}$$

Método (2D)	Trustworthiness	kNN preservados	Tiempo
PCA	0,84	11 %	< 0,1 s
t-SNE (perplexity 30)	0,98	44 %	1,7 s
UMAP (15 vecinos)	0,96	35 %	2,0 s

Idea clave

De 8 a 2 dimensiones se pierde mucho: el mejor mapa conserva menos de la mitad de los vecindarios. PCA 2D conserva lo global, no los vecinos.

Estabilidad entre semillas

UMAP, semilla 0



UMAP, semilla 1



UMAP, semilla 2



UMAP, semilla 3



- mismo dato, mismos parámetros, 4 semillas;
- vecinos compartidos con la semilla 0: **62 %** en promedio;
- el gradiente de ventas se repite; la forma del mapa no.

Un patrón que no aparece con otra semilla no existe. PCA es determinista; t-SNE y UMAP no.

Evaluación por tarea

¿El mapa conserva la información que importa? Pregunta concreta: con un k -NN ($k = 15$, CV de 5 folds), ¿se puede distinguir a los vendedores con % de retraso por encima de la mediana?

Espacio	Accuracy
Original (8 dims, escalado)	0,91
t-SNE 2D	0,84
UMAP 2D	0,82
PCA 2D	0,66

Idea clave

Si existe una variable auxiliar que importa al negocio, úsela para medir el embedding. Y recuerde que el espacio original casi siempre gana: la reducción tiene un costo.

Hands-on: vendedores de Olist

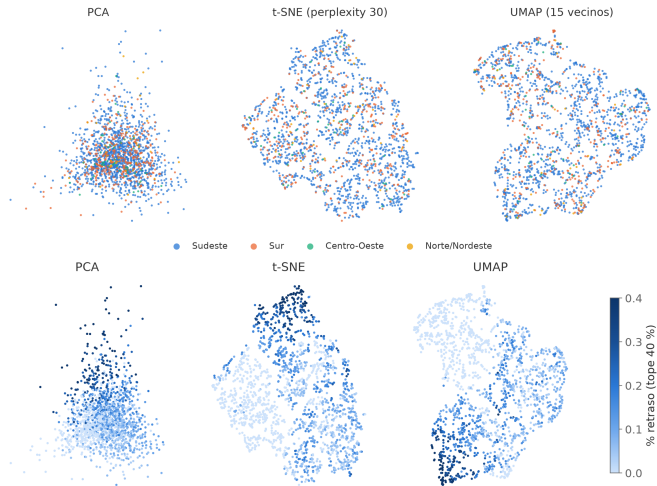
Hands - on: vendedores de Olist

Taller / tarea: Mapa de vendedores

1. En SQL sobre `olist.duckdb`: una fila por vendedor con ventas, órdenes, ticket, flete/precio, categorías, % retraso, días a la transportadora y calificación (órdenes entregadas antes de 2018-06-01, ≥ 3 órdenes).
2. `log1p` a las variables sesgadas y `StandardScaler`.
3. PCA: scree, loadings y nombre tentativo de PC1 y PC2.
4. t-SNE y UMAP en 2D; colorear por región y por % retraso.
5. Trustworthiness, vecinos preservados y 3 semillas por método.

Script de referencia: `lectures/latex/make_figures_s5_s6.py`. En el taller se hace lo mismo con clientes.

Tres mapas, un dataset



Práctica: ¿qué puede concluir?

Taller / tarea: **Situación**

Un analista muestra tres mapas de los mismos clientes: UMAP con 4 islas bien separadas, t-SNE con una docena de islas pequeñas y PCA con una sola nube. Propone lanzar 4 campañas, una por isla de UMAP.

Discuta en parejas (3 minutos):

- ¿qué se puede afirmar con estos mapas y qué no?
- ¿qué pediría antes de aprobar las campañas?
- ¿la nube única de PCA contradice a UMAP?

Idea clave: **Respuesta esperada**

Los mapas sugieren hipótesis. Hacen falta: métricas de vecindad, estabilidad entre semillas y parámetros, perfiles en las variables originales y una acción distinta por grupo. Eso es la S6.

Lo que debe quedar claro

1. El escalado y las transformaciones son parte del modelo.
2. PCA: rotación lineal que conserva la varianza global; sus ejes se pueden interpretar.
3. t-SNE y UMAP conservan vecindarios; distancias entre grupos, tamaños y densidades no significan nada.
4. Ningún embedding sin métrica (trustworthiness, vecinos) y sin probar otras semillas.
5. Ver grupos en un mapa no es tener grupos.

Idea clave: Mañana, Sesión 6

Agrupar en este espacio: K-Means, jerárquico, DBSCAN y HDBSCAN; validar sin etiquetas; outliers y drift.

Referencias

- Jolliffe, I. T. y Cadima, J. (2016). Principal component analysis: a review and recent developments. *Phil. Trans. R. Soc. A*, 374.
- van der Maaten, L. y Hinton, G. (2008). Visualizing data using t-SNE. *JMLR*, 9, 2579–2605.
- McInnes, L., Healy, J. y Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for dimension reduction. *arXiv:1802.03426*.
- Wattenberg, M., Viégas, F. y Johnson, I. (2016). How to use t-SNE effectively. *Distill*.
- Venna, J. y Kaski, S. (2001). Neighborhood preservation in nonlinear projection methods. *ICANN*.
- scikit-learn. Decomposing signals in components; Manifold learning. scikit-learn.org.

Gracias por su atención

¿Preguntas, comentarios o discusión?



Aprendizaje Automático
SI7009 · Universidad EAFIT

